

Option Pricing Models and Greeks

Documentation

Pablo López Serna

October 7, 2025

Abstract

This document describes a learning-focused repository for pricing European and American options and computing Greeks using a collection of classical and numerical models. It is intended as concise *project documentation* you can ship with the codebase and as a reference when presenting the project in a portfolio.

Contents

1	Project Overview	1
2	Repository Structure	2
3	Installation	2
4	Quick Start	2
5	Models	2
5.1	Black–Scholes (European)	2
5.2	Merton Jump–Diffusion (European)	3
5.3	American Options	3
6	Greeks	3
7	Example Script	3
8	Assumptions and Limitations	4
9	Extending the Project	4
10	Reproducibility Tips	4
11	License and Attribution	4

1 Project Overview

The project provides a minimal, educational implementation of option pricing models wrapped behind a simple example script `example.py`. It illustrates both closed-form and numerical approaches, as well as risk sensitivities (Greeks). The repository is intentionally lightweight and designed for experimentation and learning, not production trading.

Key Features

- Pricing for **European** and **American** options.
- Models: Black–Scholes, Merton Jump–Diffusion (European), Binomial Tree, Finite Difference (American), Longstaff–Schwartz Monte Carlo (American).
- Computation of core **Greeks**: Delta, Gamma, Vega, Theta, Rho.

2 Repository Structure

```
.
example.py          # Run to compare models and print Greeks
LICENSE
.gitignore
(pricer/)           # Package with pricing utils (assumed available)
```

3 Installation

Requirements

Python 3 with common scientific libraries:

```
pip install -r requirements.txt
```

If you keep the repository as a single script example, no extra setup is required. If you separate code into a `pricer` package, ensure it is on `PYTHONPATH` or installed in editable mode.

4 Quick Start

Run the example script:

```
python example.py
```

Expected behavior:

1. Fetch (or fallback) a risk-free rate.
2. Price a representative option with each model.
3. Print a side-by-side comparison and the Greeks.

5 Models

5.1 Black–Scholes (European)

Under the lognormal diffusion $dS_t = rS_t dt + \sigma S_t dW_t$, the time-0 price of a European call is

$$C = S_0 \Phi(d_1) - K e^{-rT} \Phi(d_2), \quad (1)$$

$$d_{1,2} = \frac{\ln(S_0/K) + \left(r \pm \frac{1}{2}\sigma^2\right)T}{\sigma\sqrt{T}}. \quad (2)$$

The put price follows from put–call parity: $P = C - S_0 + K e^{-rT}$.

5.2 Merton Jump–Diffusion (European)

Augment diffusion with compound Poisson jumps: dJ_t with intensity λ and lognormal jump sizes. The European price can be computed via a Poisson mixture of Black–Scholes values:

$$C_{\text{Merton}} = \sum_{n=0}^{\infty} \frac{e^{-\lambda T} (\lambda T)^n}{n!} C_{\text{BS}} \left(S_0, K, T, r - \lambda k + \frac{n\mu_J}{T}, \sqrt{\sigma^2 + \frac{n\delta_J^2}{T}} \right), \quad (3)$$

where $k = \mathbb{E}[e^Y - 1]$, $Y \sim \mathcal{N}(\mu_J, \delta_J^2)$. In practice, the sum is truncated to a small n for efficiency.

5.3 American Options

Binomial Tree. Build a recombining tree with up/down factors u, d and risk-neutral probability p . Price by backward induction, applying early-exercise at each node: $V_{i,j} = \max\{\text{intrinsic}, e^{-r\Delta t}(pV_{i+1,j+1} + (1-p)V_{i+1,j})\}$.

Finite Difference. Solve the Black–Scholes PDE

$$\frac{\partial V}{\partial t} + \frac{1}{2}\sigma^2 S^2 \frac{\partial^2 V}{\partial S^2} + rS \frac{\partial V}{\partial S} - rV = 0 \quad (4)$$

on a grid with boundary conditions and impose the early-exercise constraint $V \geq \text{intrinsic}$ (projected/penalty or PSOR methods).

Longstaff–Schwartz (LSM). Simulate price paths, regress continuation values on basis functions, and exercise when intrinsic exceeds estimated continuation. This yields a consistent estimator for American option value under simulated dynamics.

6 Greeks

For Black–Scholes, common sensitivities for a call are:

$$\Delta = \Phi(d_1), \quad \Gamma = \frac{\phi(d_1)}{S_0 \sigma \sqrt{T}}, \quad (5)$$

$$\text{Vega} = S_0 \phi(d_1) \sqrt{T}, \quad \Theta = -\frac{S_0 \phi(d_1) \sigma}{2\sqrt{T}} - rK e^{-rT} \Phi(d_2), \quad (6)$$

$$\text{Rho} = K T e^{-rT} \Phi(d_2), \quad (7)$$

where Φ and ϕ denote the standard normal CDF and PDF.

7 Example Script

Below is a shortened extract from `example.py` showing how the models are called. See the file for the full runnable version. This should be enough to replicate the behaviour.

```
from pricer.pricing import black_scholes, merton_jump, \
    american_binomial, american_finite_difference, american_lsm
from pricer.utils.helpers import get_risk_free_rate
from pricer.utils.greeks import calculate_greeks

S, K, T, sigma = 100, 100, 1.0, 0.2
option_type = "call"

try:
```

```

    r = get_risk_free_rate()
except Exception:
    r = 0.03

bs = black_scholes(S, K, T, r, sigma, option_type)
mj = merton_jump(S, K, T, r, sigma, m=1, v=0.1, lam=1, t=option_type)
am_bin = american_binomial(S, K, T, r, sigma, n_steps=100, option_type=
    option_type)
am_fd = american_finite_difference(S, K, T, r, sigma, option_type,
    n_price=100, n_time=100)
am_lsm = american_lsm(S, K, T, r, sigma, n_paths=1000, n_steps=50,
    option_type=option_type)

greeks = calculate_greeks(S, K, T, r, sigma, option_type)

```

8 Assumptions and Limitations

- Models assume frictionless markets, constant parameters (unless extended), and no arbitrage.
- Numerical methods are sensitive to grid/step choices; convergence checks are recommended.
- This repository is for **education only**; it is not investment advice.

9 Extending the Project

Ideas for future work:

- Add Heston/Stochastic Volatility or SABR models.
- Calibrate model parameters to market option surfaces.
- Provide plotting utilities to visualize price/Greeks across strikes and maturities.
- Package the `pricer` module with tests and CI.

10 Reproducibility Tips

- Fix random seeds for Monte Carlo components.
- Log the configuration: S, K, T, r, σ and discretization parameters.
- Validate American models against European prices when early exercise has no value.

11 License and Attribution

This project is released under the terms of the file `LICENSE`. If you use or adapt parts of this code, please include appropriate attribution.

Maintainer: Pablo López Serna [LinkedIn](#) [GitHub](#)